



Cimply API Reference

Writing sample — Cimply is a demonstration scheduling product for creative studios, built to showcase documentation skills. Not a live commercial product.

This reference covers the Cimply REST API: creating and managing Orders and the Statuses they move through, checking Designer availability, and looking up account Users. It assumes familiarity with the concepts covered in the *Cimply User Guide* — particularly Order Status and the Scheduler/Designer roles.

Getting Started

Base URL

```
https://api.cimply.io/v1
```

Authentication

Every request requires an API key, passed as a Bearer token in the Authorization header. Generate keys from Studio Settings → API Access (Scheduler-with-admin permission required).

```
GET /v1/orders
Authorization: Bearer cimply_live_a1b2c3d4e5f6
```

Requests without a valid key return 401 Unauthorized. Keys are scoped to a single studio account and inherit the permissions of the user who created them — a key created by a Designer cannot perform Scheduler-only actions (see Who Can Move What in the Orders section).

Content Type

Send and expect `application/json`. Include a `Content-Type: application/json` header on requests with a body.

Pagination

List endpoints are paginated with `page` and `per_page` query parameters (default `per_page=25`, max 100). Responses include a meta object:

```
{
  "data": [ ... ],
  "meta": {
    "page": 1,
    "per_page": 25,
    "total_count": 118
  }
}
```



Rate Limits

Each API key is limited to 120 requests per minute. Responses include X-RateLimit-Remaining and X-RateLimit-Reset headers. Exceeding the limit returns 429 Too Many Requests with a Retry-After header (seconds).

Errors

Errors return a JSON body with a machine-readable code and a human-readable message:

```
{
  "error": {
    "code": "invalid_status_transition",
    "message": "Orders can only move to Complete after a time entry has been logged.",
    "status": 409
  }
}
```

Each resource section below lists the status codes specific to its endpoints, in addition to these account-wide errors:

Status	Meaning
400	Malformed request — invalid JSON or an unrecognized parameter.
401	Missing, expired, or invalid API key.
429	Rate limit exceeded.
500	Unexpected server error. Safe to retry with backoff.

Orders

An Order is the basic unit of work in Cimply — a single piece of client work, from brief to completion, along with its Status, assignment, and logged time. This section documents the Order object and every endpoint that reads or changes it.

The Order Object

Field	Type	Description
id	string	Unique identifier, e.g. ord_8f2ac1.
title	string	40 characters or fewer.
description	string	The work brief — what's needed, references, constraints.
estimated_hours	number	Scheduler's time estimate at creation.
actual_hours	number null	Sum of logged time entries. Null until the assignee logs time.



Field	Type	Description
status	object	The Order's current status, expanded inline as { id, name, type }. See Statuses below.
scheduler_id	string	User ID of the Order's owning Scheduler.
designer_id	string null	User ID of the assigned Designer. Null if unassigned.
scheduled_start	string	ISO 8601 timestamp.
scheduled_end	string	ISO 8601 timestamp.
past_due	boolean	Computed. True if scheduled_end has passed and status.type is non-terminal.
late_start	boolean	Computed. True if scheduled_start has passed while status.type is new or assigned.
created_at	string	ISO 8601 timestamp.
updated_at	string	ISO 8601 timestamp.

Status Types

Every status an account configures maps to one of seven fixed types. Types are what actually drive Cimply's business rules — permissions, terminal locking, and the Past-Due/Late Start flags — so they're built into the platform and can't be renamed, added, or removed, even as accounts customize their own status names and colors:

Type	Meaning	Terminal ?
new	Created, not yet assigned to a Designer.	No
assigned	A Designer is attached; work hasn't started.	No
in_progress	Designer is actively working.	No
hold	Paused, waiting on something.	No
submitted	Designer submitted; Scheduler needs to look at it.	No
complete	Closed successfully.	Yes
cancelled	Closed without completion.	Yes

Statuses

A Status is the named, colored, per-account object that actually appears on Orders and in the product UI — each one maps to exactly one Type above. A Scheduler manages these from Studio Settings → Statuses.



Field	Type	Description
id	string	Unique identifier, e.g. sts_6f7a8b.
account_id	string	Studio account this status belongs to.
name	string	Display label, e.g. "In Progress". Customizable.
type	string	One of the seven fixed types above. Not customizable.
color	string	Hex color shown in the product UI, e.g. #732EB5.
is_default	boolean	Unique per type per account, enforced at the database level. Marks which status of that type is used automatically — for example, a new Order's starting status.
sort_order	integer	Controls display order only; no effect on behavior.
created_by	string	User ID that created this status.
created_at	string	ISO 8601 timestamp.
last_modified_by	string	User ID of the most recent edit.
last_modified_at	string	ISO 8601 timestamp.

In P0, every account has exactly one Status per Type — `is_default` being unique per type per account means a type and its status are effectively 1:1. P1 lifts that constraint for at least some types, letting accounts add multiple named statuses that share a type (several flavors of `hold` with different names and colors, for instance) while the same type-based rules keep governing them. The API is already shaped for that: Orders reference a status by `id`, never by type directly, so nothing about the Order or Change Order Status contracts below has to change when P1 ships.

List Statuses

GET /statuses

Returns every status configured for the account, ordered by `sort_order`. In P0 this is always the same seven rows, one per type.



```
{
  "data": [
    { "id": "sts_1a2b3c", "name": "New", "type": "new", "color": "#9E9E9E", "is_default": true, "sort_order": 1 },
    { "id": "sts_2b3c4d", "name": "Assigned", "type": "assigned", "color": "#546E7A", "is_default": true, "sort_order": 2 },
    { "id": "sts_3c4d5e", "name": "In Progress", "type": "in_progress", "color": "#1565C0", "is_default": true, "sort_order": 3 },
    { "id": "sts_4d5e6f", "name": "On Hold", "type": "hold", "color": "#8F6B00", "is_default": true, "sort_order": 4 },
    { "id": "sts_5e6f7a", "name": "Needs Review", "type": "submitted", "color": "#732EB5", "is_default": true, "sort_order": 5 },
    { "id": "sts_6f7a8b", "name": "Complete", "type": "complete", "color": "#2E7D32", "is_default": true, "sort_order": 6 },
    { "id": "sts_7a8b9c", "name": "Cancelled", "type": "cancelled", "color": "#B00020", "is_default": true, "sort_order": 7 }
  ]
}
```

List Orders

GET /orders

Returns Orders visible to the authenticated user — all Orders for Schedulers, only assigned Orders for Designers.

Parameter	Type	Description
status_id	string	Filter to Orders with this exact status.
type	string	Filter to Orders whose status matches this type — useful even once an account has multiple statuses sharing a type.
designer_id	string	Filter to Orders assigned to a Designer.
scheduler_id	string	Filter to Orders owned by a Scheduler.
past_due	boolean	Filter to Orders with the Past-Due flag.
start_date	string	Only Orders with scheduled_start on or after this date (YYYY-MM-DD).
end_date	string	Only Orders with scheduled_end on or before this date (YYYY-MM-DD).
page	integer	Page number. Default 1.
per_page	integer	Results per page. Default 25, max 100.

Example request:

```
GET /v1/orders?type=in_progress&designer_id=usr_77b4
```

Example response — 200 OK:



```
{
  "data": [
    {
      "id": "ord_8f2ac1",
      "title": "Homepage hero banner refresh",
      "status": { "id": "sts_3c4d5e", "name": "In Progress", "type":
"in_progress" },
      "scheduler_id": "usr_2e91",
      "designer_id": "usr_77b4",
      "scheduled_start": "2026-08-03T09:00:00Z",
      "scheduled_end": "2026-08-05T17:00:00Z",
      "past_due": false,
      "late_start": false
    }
  ],
  "meta": { "page": 1, "per_page": 25, "total_count": 1 }
}
```

Create Order

POST /orders

Scheduler only. Creates a new Order with the account's default status for type new — or type assigned if designer_id is included.

Field	Required	Description
title	Yes	40 characters or fewer.
description	Yes	The work brief.
estimated_hours	Yes	Scheduler's time estimate.
scheduled_start	Yes	ISO 8601 timestamp.
scheduled_end	Yes	ISO 8601 timestamp. Must be after scheduled_start.
designer_id	No	Assigns a Designer at creation. A Scheduler may assign themselves.

Example request:



```
POST /v1/orders
Content-Type: application/json

{
  "title": "Homepage hero banner refresh",
  "description": "Swap the summer campaign hero for the fall lineup. Sizes and copy attached in the brief link.",
  "estimated_hours": 6,
  "scheduled_start": "2026-08-03T09:00:00Z",
  "scheduled_end": "2026-08-05T17:00:00Z"
}
```

Returns 201 Created with the new Order object. When `designer_id` is set, the Designer receives an assignment email within a few minutes, matching in-app assignment behavior.

Get Order

GET /orders/{order_id}

Returns a single Order. 404 if the Order doesn't exist or isn't visible to the authenticated user.

Update Order

PATCH /orders/{order_id}

Updates editable fields on a non-terminal Order: `title`, `description`, `estimated_hours`, `scheduled_start`, `scheduled_end`, and `designer_id` (reassignment).

Reassignment via `designer_id` is only permitted while `status.type` is `new` or `assigned`. For an Order whose `status.type` is `in_progress`, `hold`, or `submitted`, reassignment returns 409 Conflict — cancel the Order and create a new one for the incoming Designer instead, which keeps the original Order's history intact.

To change an Order's status, use Change Order Status below rather than PATCH — status transitions carry permission and validation rules that a plain field update doesn't.

Change Order Status

POST /orders/{order_id}/status

Moves an Order to a different status. The body takes the target `status_id`. Because P0 guarantees exactly one status per type per account, most integrations resolve the id once via List Statuses and cache the type → id mapping, or query GET /statuses?type=complete when they only know the type they want.

Whether a role can make this change is enforced by the target status's type, not its account-specific id or name — so this table stays valid even after a studio renames a status, or (post-P1) adds a second status within the same type:



Target Type	Scheduler	Designer
new / assigned	✓	—
in_progress	✓	✓
hold	✓	✓
submitted	✓	✓
complete	✓	—
cancelled	✓	—

Example request:

```
POST /v1/orders/ord_8f2ac1/status
Content-Type: application/json

{ "status_id": "sts_6f7a8b" }
```

This endpoint returns 409 Conflict when:

- The authenticated user's role can't set a status of that type (see table above).
- The target type is not reachable from the current type — for example, moving directly from `new` to `complete`.
- The target type is `complete` but the Order has no logged time entry (see Log Time below).
- The Order's current status is already type `complete` or `cancelled` — both are terminal and locked.

Log Time

POST /orders/{order_id}/time_entries

Adds a time entry from the Order's assignee. At least one entry is required before an Order can move to `complete` — it becomes the Order's `actual_hours` total.

Field	Required	Description
hours	Yes	Decimal hours, e.g. 3.5.
note	No	Optional context for the entry.

```
POST /v1/orders/ord_8f2ac1/time_entries
Content-Type: application/json

{ "hours": 3.5, "note": "First pass on hero composition" }
```

Order-Specific Errors



Status	Code	Meaning
403	forbidden_status_transition	Authenticated user's role can't set a status of that type.
404	order_not_found	No Order with that ID, or it isn't visible to this user.
404	status_not_found	status_id doesn't exist, or doesn't belong to this account.
409	invalid_status_transition	Target status's type isn't reachable from the current type.
409	time_entry_required	Attempted to move to a complete-type status with no logged time.
409	reassignment_blocked	Attempted to reassign an Order whose status type is past new/assigned.
422	invalid_date_range	scheduled_end is not after scheduled_start.
422	title_too_long	title exceeds 40 characters.

Webhooks

Configure webhook URLs in Studio Settings → API Access. Cimply sends a signed POST for each event:

Event	Fires when
order.created	A new Order is created.
order.status_changed	An Order's status changes, including automatically-cleared flags.
order.time_logged	A time entry is added to an Order.

Example payload:

```
{
  "event": "order.status_changed",
  "order_id": "ord_8f2ac1",
  "from_status": { "id": "sts_3c4d5e", "type": "in_progress" },
  "to_status": { "id": "sts_5e6f7a", "type": "submitted" },
  "changed_by": "usr_77b4",
  "occurred_at": "2026-08-04T15:12:00Z"
}
```

Availability

Availability blocks record when a Designer is free to take on work. Schedulers reference these when creating or assigning Orders; Cimply doesn't currently prevent double-booking automatically, so this is informational rather than enforced.



The Availability Object

Field	Type	Description
id	string	Unique identifier, e.g. avl_4c19f0.
designer_id	string	User ID of the Designer.
date	string	YYYY-MM-DD.
start_time	string	24-hour HH:MM, in the Designer's studio time zone.
end_time	string	24-hour HH:MM.

List Availability

GET /availability

Accepts designer_id, start_date, and end_date query parameters (all optional; omitting designer_id returns blocks across the studio).

```
GET /v1/availability?designer_id=usr_77b4&start_date=2026-08-01&end_date=2026-08-07
```

```
{
  "data": [
    {
      "id": "avl_4c19f0",
      "designer_id": "usr_77b4",
      "date": "2026-08-04",
      "start_time": "09:00",
      "end_time": "17:00"
    }
  ]
}
```

Create Availability Block

POST /availability

Designers create their own blocks; a Scheduler may create one on behalf of a Designer by including designer_id. Returns 201 Created with the new object, or the account-wide error responses described in Getting Started.



```
POST /v1/availability
Content-Type: application/json
```

```
{
  "date": "2026-08-06",
  "start_time": "09:00",
  "end_time": "13:00"
}
```

Users

Users hold the Scheduler and Designer roles described in the User Guide. Admin isn't a separate role value here — it's an `is_admin` permission flag on a Scheduler user, matching the product's own design: Cimply has no separate Admin dashboard or persona, just a Scheduler with account-management permissions.

The User Object

Field	Type	Description
<code>id</code>	string	Unique identifier, e.g. <code>usr_77b4</code> .
<code>name</code>	string	Full name.
<code>email</code>	string	Account email.
<code>role</code>	string	<code>scheduler</code> or <code>designer</code> .
<code>is_admin</code>	boolean	True if this Scheduler also has account-management permissions. Always false for designer users.
<code>created_at</code>	string	ISO 8601 timestamp.

List Users

GET /users

Accepts an optional `role` query parameter (`scheduler` or `designer`) to filter results.

```
GET /v1/users?role=designer
```

Get User

GET /users/{user_id}

Returns a single User object. 404 if the user isn't on the authenticated account's studio.